

AI-Orchestrator

Gateway corporativo multi-agente com LLM **100% on-premise**. Uma pergunta em linguagem natural é roteada, executada em paralelo por agentes especialistas sobre microserviços de domínio e sintetizada em uma resposta única — sem nenhum token saindo do perímetro da empresa.

ORQUESTRAÇÃO

LangGraph · Qdrant (rota semântica)

INFERÊNCIA

Ollama · Qwen (local, GPU)

DOMÍNIOS

Finanças · RH · Estoque · Vendas

CUSTO POR CONSULTA

R\$ 0 em API

Perguntas de negócio cruzam sistemas. Pessoas não deveriam precisar cruzá-los manualmente.

Dados fragmentados

"Posso aceitar um pedido de 500 unidades com 15% de desconto?" exige consultar vendas (política), estoque (saldo) e finanças (caixa) — três sistemas, três telas, três equipes.

LLM em nuvem = dado fora

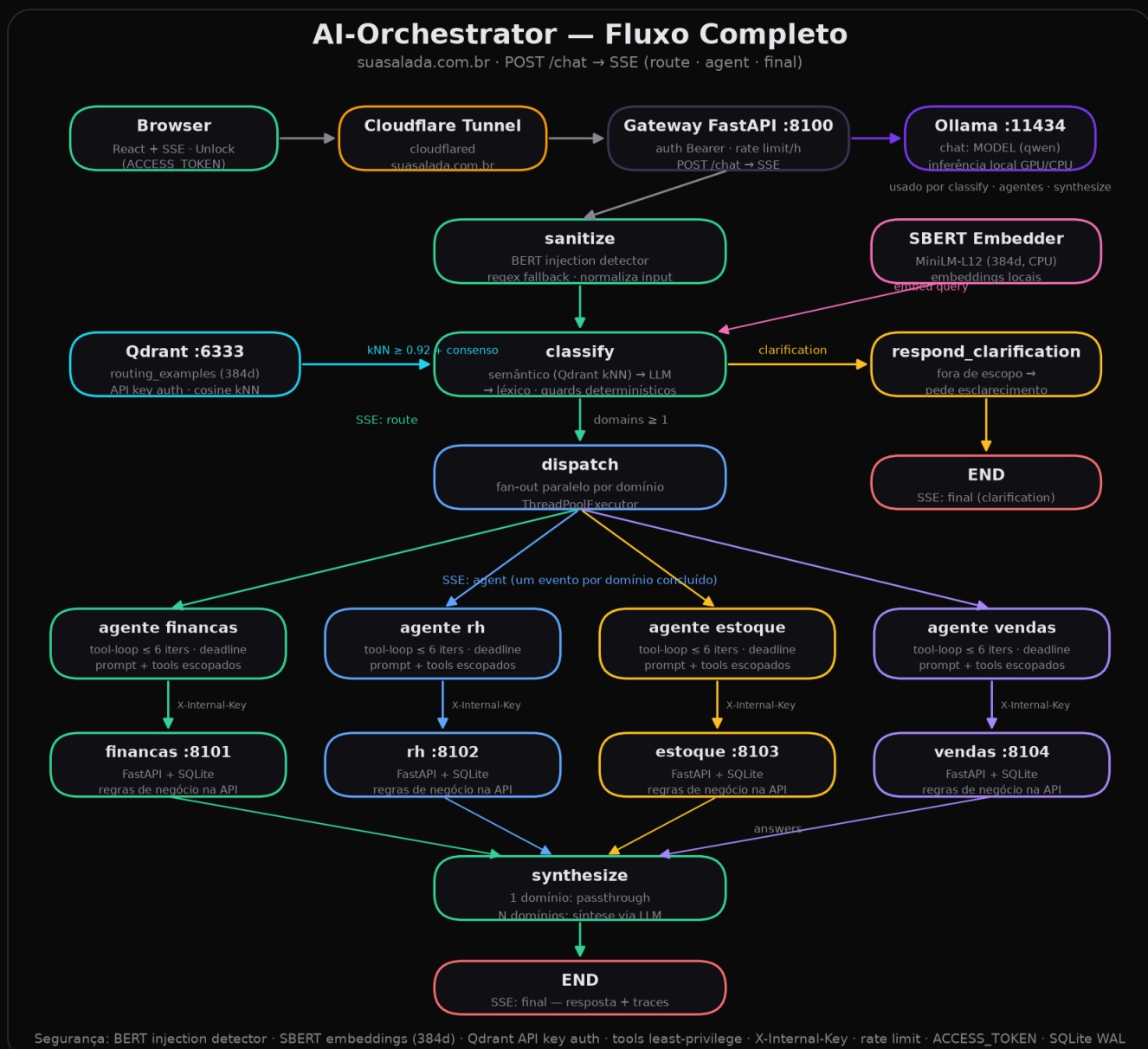
Conectar ChatGPT/Claude direto aos ERPs envia salários, margens e fluxo de caixa para um terceiro. Risco LGPD e custo por token que cresce com o uso.

Regra de negócio no prompt

Abordagens ingênuas codificam alçadas e limites no prompt — o modelo "decide" regra de negócio e alucina valores. Auditoria impossível.

Fluxo completo — da pergunta à resposta auditável

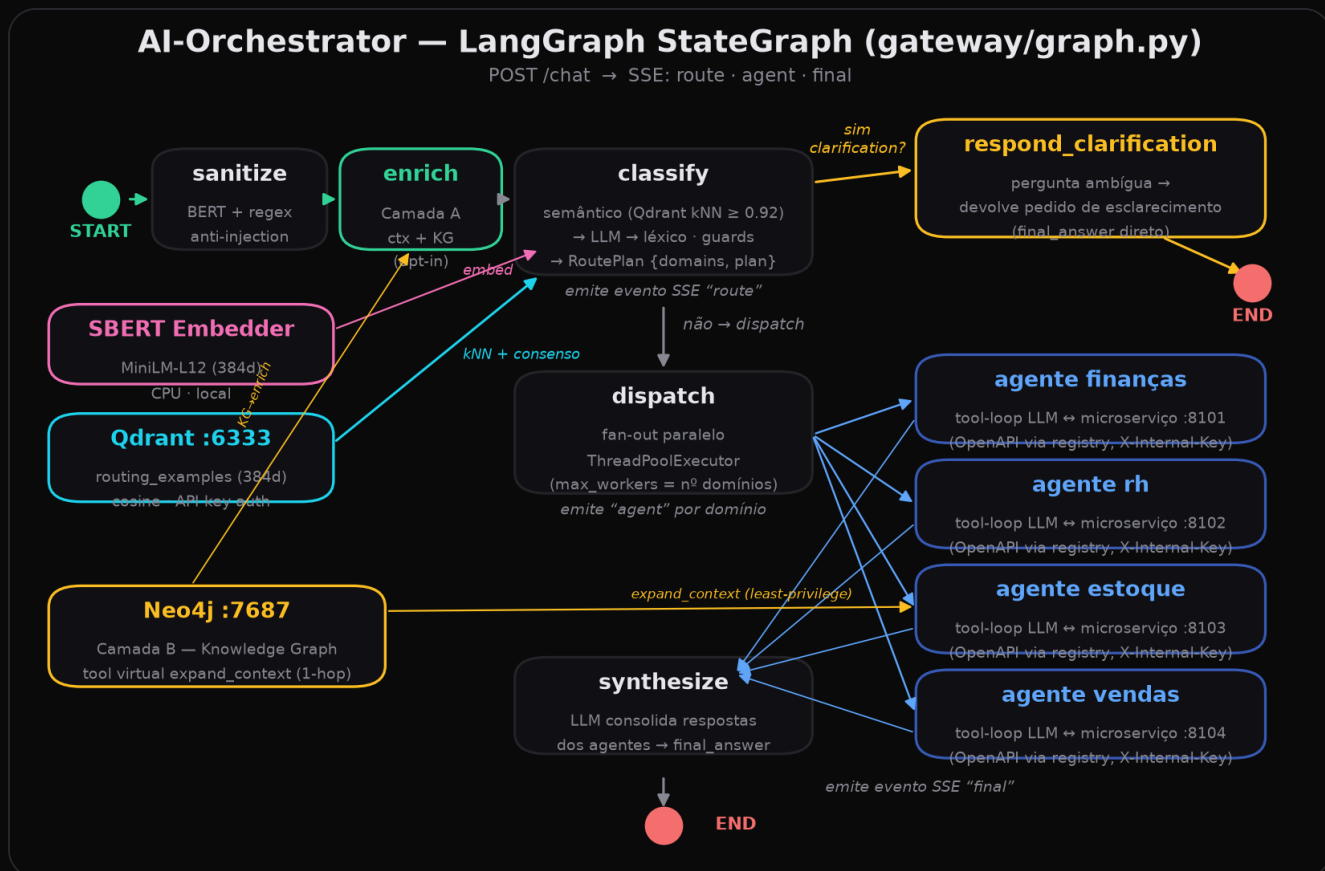
Browser → Cloudflare Tunnel → Gateway FastAPI → grafo LangGraph → agentes em paralelo → microserviços com as regras → síntese. Progresso transmitido por SSE em tempo real (route · agent · final).



Arquitetura end-to-end em produção. Inferência local via Ollama — nenhum dado sai do perímetro.

LangGraph: 5 nós, decisões explícitas

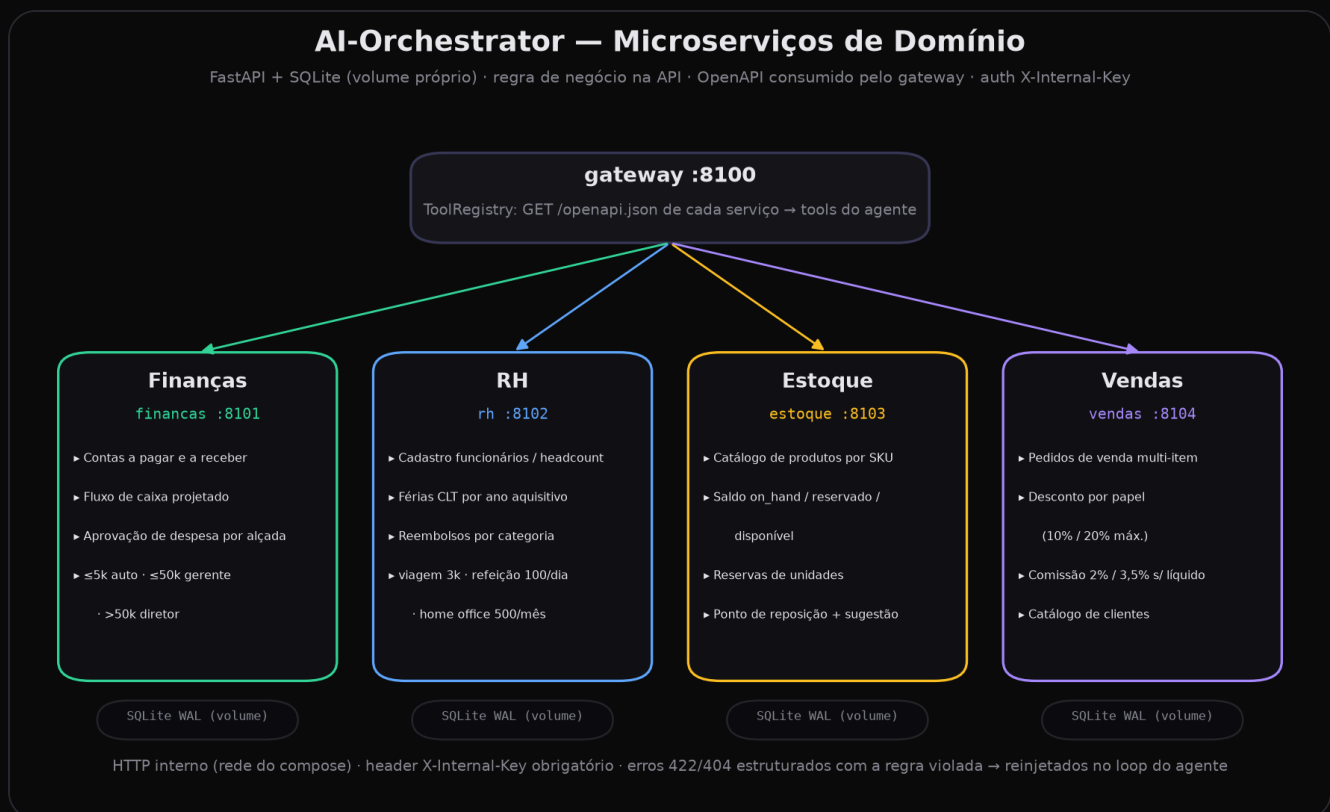
Sanitização anti-injeção → classificação em três camadas (semântica via Qdrant kNN, LLM com retry, fallback léxico determinístico) → fan-out paralelo (ThreadPoolExecutor) → síntese. Pergunta já conhecida resolve em ~0,1 s por similaridade vetorial, sem chamar o LLM. Fora de escopo? O grafo pede esclarecimento em vez de alucinar.



StateGraph do gateway (gateway/graph.py) — cada transição é observável e testada.

A regra de negócio vive na API — nunca no prompt

Quatro microserviços FastAPI independentes, cada um com seu banco e suas regras (alçadas, CLT, reposição, descontos). O LLM consulta e executa; quem valida é o serviço. Erro 422 volta estruturado com a regra violada e reentra no loop do agente.



Gateway descobre as capacidades de cada serviço via OpenAPI — adicionar um domínio novo não muda o orquestrador.

Um agente por domínio, least-privilege por construção

Mesmo modelo residente, mas cada agente só enxerga as tools do seu microserviço. Loop limitado (≤ 6 iterações + deadline) e toda afirmação numérica obrigatoriamente rastreável a uma chamada de tool.

Agente Finanças

serviço :8101

Contas a pagar/receber · fluxo de caixa projetado · despesas · alçada de aprovação

FERRAMENTAS (OpenAPI → tools)

- ▶ `list_accounts` / `create_account` — contas a pagar e receber
- ▶ `settle_account` — `pay` (pagar) / `receive` (receber)
- ▶ `cashflow` — projeção de caixa por período

REGRAS DE NEGÓCIO (vivem na API, não no prompt)

- ▶ $\leq$ R\$ 5.000 → auto-aprovada
- ▶ $\leq$ R\$ 50.000 → exige aprovador 'gerente'
- ▶ $>$ R\$ 50.000 → exige aprovador 'diretor'
- ▶ `pay` só em conta 'pagar' aberta; `receive` só em 'receber'

COMPORTAMENTO DO SUBAGENTE

- Mesmo LLM residente (Ollama) com system prompt escopado ao domínio
- Tools geradas do OpenAPI do microserviço (ToolRegistry, least-privilege)
- Loop de tool-calling: máx. 6 iterações + deadline; 422/404 reinjetado como observação
- Toda afirmação numérica vem do retorno de uma tool — nunca inventa valores
- Chamadas HTTP autenticadas com X-Internal-Key

Agente RH

serviço :8102

Funcionários · férias CLT · reembolsos · headcount

FERRAMENTAS (OpenAPI → tools)

- ▶ `list_employees / get_employee` — cadastro e salários
- ▶ `vacation_balance / request_vacation` — férias por ano aquisitivo
- ▶ `create_reimbursement` — viagem, refeição, home office

REGRAS DE NEGÓCIO (vivem na API, não no prompt)

- ▶ 30 dias de férias após 12 meses de admissão
- ▶ fracionamento limitado a N períodos por ano aquisitivo
- ▶ reembolso: $\text{viagem} \leq R3.000 \cdot \text{refeição} \leq R100/\text{dia}$
- ▶ home office $\leq R\$500/\text{mês}$

COMPORTAMENTO DO SUBAGENTE

- Mesmo LLM residente (Ollama) com system prompt escopado ao domínio
- Tools geradas do OpenAPI do microserviço (ToolRegistry, least-privilege)
- Loop de tool-calling: máx. 6 iterações + deadline; 422/404 reinjetado como observação
- Toda afirmação numérica vem do retorno de uma tool — nunca inventa valores
- Chamadas HTTP autenticadas com X-Internal-Key

Agente Estoque

serviço :8103

Produtos por SKU · saldo disponível · reservas · ponto de reposição

FERRAMENTAS (OpenAPI → tools)

- ▶ `list_products / get_product` — catálogo e saldo por SKU
- ▶ `create_reservation / release` — reserva de unidades
- ▶ `replenishment` — sugestão de compra

REGRAS DE NEGÓCIO (vivem na API, não no prompt)

- ▶ $\text{available} = \text{on_hand} - \text{reserved}$
- ▶ reserva nunca excede o disponível
- ▶ reposição quando $\text{available} \leq \text{reorder_point}$
- ▶ sugestão de compra $= \text{reorder_point} \times 2 - \text{available}$

COMPORTAMENTO DO SUBAGENTE

- Mesmo LLM residente (Ollama) com system prompt escopado ao domínio
- Tools geradas do OpenAPI do microserviço (ToolRegistry, least-privilege)
- Loop de tool-calling: máx. 6 iterações + deadline; 422/404 reinjetado como observação
- Toda afirmação numérica vem do retorno de uma tool — nunca inventa valores
- Chamadas HTTP autenticadas com X-Internal-Key

Agente Vendas

serviço :8104

Pedidos de venda · política de desconto por papel · comissão

FERRAMENTAS (OpenAPI → tools)

- ▶ `create_order` — pedido com itens, papel e desconto
- ▶ `list_orders` / `get_order` — consulta de pedidos

REGRAS DE NEGÓCIO (vivem na API, não no prompt)

- ▶ `desconto` $\leq 10\%$ 'vendedor' · $\leq 20\%$ 'gerente'
- ▶ nunca acima de 20% (hard limit)
- ▶ comissão: 2% até R\$ 10.000 líquido · 3,5% acima

COMPORTAMENTO DO SUBAGENTE

- Mesmo LLM residente (Ollama) com system prompt escopado ao domínio
- Tools geradas do OpenAPI do microserviço (ToolRegistry, least-privilege)
- Loop de tool-calling: máx. 6 iterações + deadline; 422/404 reinjetado como observação
- Toda afirmação numérica vem do retorno de uma tool — nunca inventa valores
- Chamadas HTTP autenticadas com X-Internal-Key

Defesa em camadas, validada por evals

Boundary público

- ▶ ACCESS_TOKEN no /chat
- ▶ Rate limit por hora
- ▶ Cloudflare Tunnel — zero porta aberta
- ▶ Serviços internos só em loopback

Anti prompt-injection

- ▶ Nó sanitize neutraliza markup/controle
- ▶ Classificador treinado a ignorar comandos injetados
- ▶ Eval de injeção com golden set versionado

Perímetro interno

- ▶ X-Internal-Key em todo serviço
- ▶ Tools least-privilege por domínio
- ▶ Circuit breaker e timeouts
- ▶ Trace completo de tool-calls por resposta

Stack 100% open-source — zero licença, zero vendor lock-in

Python · FastAPI

LangGraph

Ollama

Qwen 3.5-9B LoRA (fine-tuned)

Qdrant (busca vetorial)

SBERT (MiniLM-L12-v2)

BERTimbau (injection detector)

SQLite

React + Vite

SSE

Docker Compose

Cloudflare Tunnel

pytest + evals (golden sets)

Qualidade contínua

Suíte de testes por camada + evals versionados de roteamento, domínios e injeção — regressão de comportamento do LLM é detectada como bug, não como surpresa.

Modelo configurável + LoRA

Variável MODEL troca o modelo sem código. Produção: **Qwen 3.5-9B LoRA** (fine-tuned em 3.050 exemplos, 100% GPU, ~2–4 s/task). Mesmo grafo, mesmo contrato.

Extensível por OpenAPI

Novo domínio = novo microserviço com OpenAPI. O ToolRegistry gera as tools automaticamente; o orquestrador não muda.

SBERT Embeddings — roteamento semântico em CPU

Embeddings dedicados para o Semantic Router, rodando inteiramente em CPU e liberando a GPU para inferência LLM.

paraphrase-multilingual-MiniLM-L12-v2

- ▶ 384 dimensões, CPU-only
- ▶ Substituiu nomic-embed-text (768 dim, GPU)
- ▶ 0.02 s vs 0.84 s LLM — **41x mais rápido**
- ▶ Multilingual: pt-BR, en, es sem troca de modelo

Embedder Protocol

- ▶ Interface abstrata `EmbedderProtocol`
- ▶ `SBERTEEmbedder` (primário, CPU)
- ▶ `OllamaEmbedder` (fallback, GPU)
- ▶ Troca de backend sem alterar o router

Injection Detector — BERTimbau fine-tunado

Classificador binário treinado sobre BERTimbau para detectar e bloquear tentativas de prompt injection antes que a query chegue ao LLM.

Modelo

- ▶ neuralmind/bert-base-portuguese-cased
- ▶ Fine-tunado: 3 epochs, lr=2e-5, batch=16
- ▶ Dataset sintético: 400 exemplos (200 clean + 200 injection)
- ▶ **100% validation accuracy**

Pipeline

- ▶ sanitize → BERT score → flag/block
- ▶ Fallback regex quando modelo indisponível
- ▶ Score threshold configurável
- ▶ Logging estruturado de tentativas bloqueadas

Resultados — gates de qualidade validados

90.5%

Routing accuracy: 57/63 queries roteadas corretamente. Gate 90% — **PASS**.

0 leaks

Injection eval: 0/6 tentativas de injection passaram. Gate — **PASS**.

0 / 0

Auditoria /hm-engineer: zero achados CRITICO, zero ALTO. Score gap filter + consenso unânime.

Golden set: 64 exemplos (20 novos multi-domínio). Evals com score gap filter e consenso unânime entre classificadores para evitar falso-positivos.

Arquitetura — SBERT em CPU, Qwen em GPU

Dois modelos BERT rodando em CPU, sem competir com o LLM por VRAM. Pipeline completo de segurança e roteamento antes da inferência generativa.

User Query

|

▼

Sanitize — BERTimbau injection classifier (CPU)

| clean? → continue | injection? → block

▼

Semantic Router — SBERT + Qdrant kNN (CPU)

| high confidence? → skip LLM

▼

LLM Classifier — Qwen 3.5-9B LoRA (GPU)

|

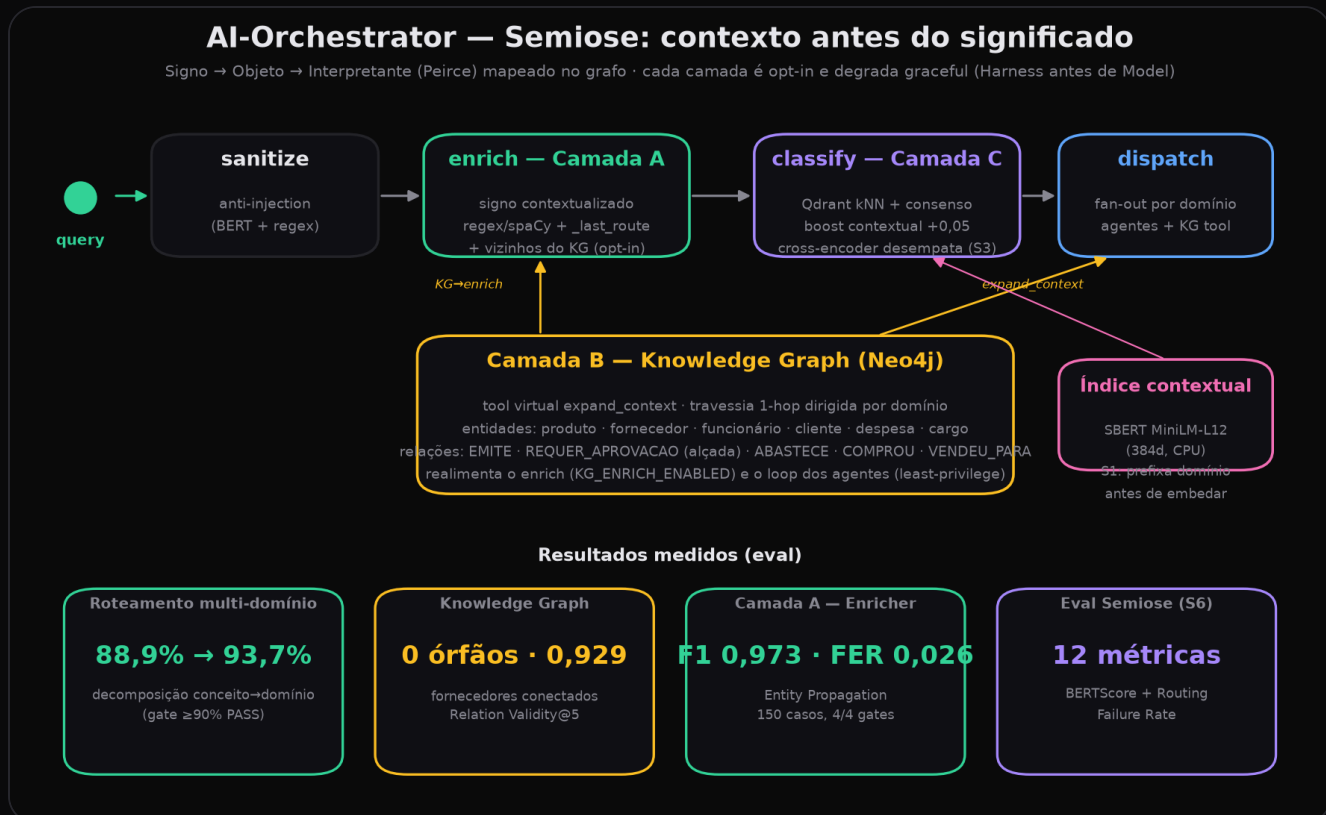
▼

Agents — fan-out paralelo por domínio (GPU)

SBERT e BERTimbau ocupam ~500 MB de RAM. GPU (12 GB VRAM) fica 100% dedicada ao Qwen. Queries já conhecidas resolvem em ~0.02 s pelo Semantic Router sem acionar o LLM.

Semiose — contexto antes do significado

Camada inspirada na semiose triádica de Peirce (Signo → Objeto → Interpretante), mapeada no grafo. Três camadas opt-in que degradam graceful: enriquecer a query com contexto, expandir relações em um grafo de conhecimento e re-ranquear a rota semântica.



Camada A (enrich) · Camada B (Knowledge Graph em Neo4j, tool virtual) · Camada C (re-rank com cross-encoder) — cada uma controlada por flag.

Camada A — Enrich

Contexto estruturado (domínio anterior + entidades) sem chamar LLM. Realimenta com vizinhos do grafo. Entity Propagation F1 **0,973**.

Camada B — Knowledge Graph

Neo4j como tool virtual expand_context (least-privilege). Alçadas de aprovação, fornecedores e estoque conectados — **0 órfãos**, Relation Validity@5 **0,929**.

Roteamento multi-domínio

Decomposição conceito→domínio + permissão→clarification no classifier. Acurácia **88,9%** → **93,7%** (gate ≥90% PASS).

On-premise: custo marginal por consulta → zero

Cada consulta multi-domínio dispara 8–15 chamadas de LLM (classificação, loops de tools, síntese) — em API de nuvem, isso é caro e cresce linearmente com o uso.

CENÁRIO (1.000 CONSULTAS/MÊS)	CUSTO MENSAL ESTIMADO*	DADOS SAEM DO PERÍMETRO?
API frontier (GPT-4o / Claude Sonnet) ~40k tokens por consulta multi-domínio	R\$ 800 – 1.600	Sim
GPU em nuvem 24/7 (A10G, g5.xlarge ~US\$ 1/h)	~R\$ 4.000	Não (mas infra de terceiro)
Esta arquitetura — GPU própria (RTX 3060 12 GB): energia + amortização	~R\$ 100 (50 energia + 50 amortização)	Não

R\$ 0

de API e de licença de software. Stack inteira open-source; Cloudflare Tunnel no plano gratuito; domínio R\$ 40/ano.

~R\$ 100/mês

TCO mensal amortizado: energia + hardware (~R\$ 1.800 ÷ 36 meses). 8–16× mais barato que API frontier; ~40× que GPU em nuvem.

LGPD ✓

salários, margens e caixa nunca deixam a rede. Sem DPA com terceiros, sem risco de retenção de prompt, custo de compliance reduzido.

* Estimativas com câmbio R\$ 5,30/US\$. API: 40k tokens/consulta (medido na demo com fan-out de 3 domínios) a preços públicos de GPT-4o (US\$ 2,50/M input, US\$ 10/M output). Energia: ~170 W de GPU em uso intermitente, tarifa média residencial. Amortização: RTX 3060 (~R\$ 1.800) linear em 36 meses de vida útil, valor residual zero. O custo de API escala linearmente com o volume; o on-premise é praticamente flat até a saturação da GPU — quanto maior o uso, maior a economia.

IA corporativa com soberania de dados, custo previsível e regra de negócio auditável.

Tese arquitetural

LLM como orquestrador de capacidades, não como dono da verdade: o modelo conversa, as APIs decidem. Isso elimina alucinação de regra e torna cada resposta rastreável.

Tese econômica

Para cargas internas e dados sensíveis, inferência local em hardware commodity vence nuvem em custo total já no primeiro mês — e a economia cresce com o uso.

Em produção

Live em **suasalada.com.br** — gateway, 4 domínios, streaming SSE e tunnel, rodando em uma única máquina com RTX 3060.